Appendix A: Parameters for SimP Dataset

The perturbed parameters are written as $\tilde{\boldsymbol{\theta}} = \boldsymbol{\theta} + \boldsymbol{p}$, where $\tilde{\boldsymbol{\theta}}, \boldsymbol{\theta}$, and $\boldsymbol{p}$ refer to perturbed parameter, original parameter, and perturbation vector, respectively. We set the standard deviation of $\boldsymbol{p}$ after experimenting with several possible values to explore how it influences walking success rate. In Figure 1, successful walking episodes end with final z coordinates of the head camera above 0.7 units (measured in meters), while falling episodes have final z coordinates near 0. As a result, we selected the following standard deviations: $[0.01, 0.015, 0.02, 0.025, 0.03, 0.09, 0.095, 0.1, 0.105, 0.11]$, where the last five are more likely to produce falling episodes, while the vectors sampled from the first five have negligible impact on the successful walking trajectory. Profiles for six representative joints, averaged over all episodes in our simulated dataset, are shown in **??**. These profiles constitute complete episodes; each $\boldsymbol{X}$ is a short clip extracted from a longer complete episode.

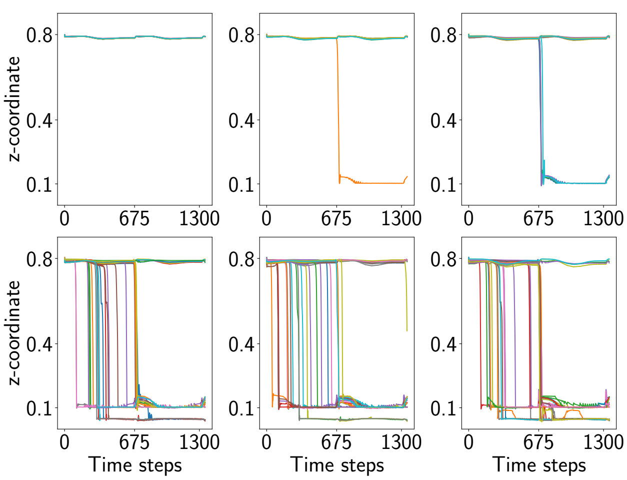


Figure 1: Effects of perturbation standard deviations. Each panel contains 30 episodes (shown in different colors); panels refer to standard deviation of [0.01, 0.02, 0.03, 0.09, 0.10, 0.11], left to right, top to bottom. The horizontal axis is time and the vertical axis is the z-coordinate of the head (which should be stable around 0.8 unless a fall occurs).

Appendix B: Hyper-parameters and Implementation Details

Our implementation uses Pytorch (**?**) and scikit-learn (**?**). All encoders are optimized using Adam (**?**). The SVM uses the linear kernel with L2 penalty. Based on theorem 3 in (**?**), the regularization parameter is set to approximately the square root of the inverse of training samples. All experimental results are averaged over three random seeds, chosen arbitrarily but fixed for reproducibility: 608, 1247, 3224. All encoders are trained for 10k iterations. The regularization parameter of LinearSVC is set to 3×10^{-2} for the RealP dataset and 5×10^{-3} for the SimP dataset.

Motion Encoder

Our motion encoder is trained with a learning rate of 3×10^{-4} for the first 4k iterations and a weight decay of 10^{-4} on both datasets. Then, the learning rate drops to 10^{-5}. Batch size is set to 64 for both datasets.

Vision Encoder

The learning rate for our vision encoder is set to 10^{-4} and decays 20% every 2.5k iterations for the SimP dataset and every 1k iterations for the RealP dataset. Batch size is set to 16 for both datasets. Details of our vision models are listed in Table 1.

Trajectory Encoder

Our trajectory model employs two MLPs as the encoder and decoder and is trained with a learning rate of 5×10^{-7} and a 10^{-5} weight decay. To fit sequences of different lengths, the number of layers is dynamically determined by the length of input sequences. The number of output channels in linear layers is half of the number of input channels (always rounded to lower integer if necessary) and doubled in encoder and decoder. Hence, the extracted feature lengths for the RealP and SimP datasets are 100 and 300, respectively. Batch size is set to 64 for both datasets.

Appendix C: Quantitative Analysis of the Joint Angle Prediction

The MSE loss of our motion encoder, after training with various learning rates, is shown in Figure 2. The motion encoder is trained to regress 1D joint angles at a prediction span of 30 on SimP and 12 on RealP. Extensive experiments, wherein relative positions are taken as motion inputs, are conducted on the SimP dataset. While keeping the prediction span for training fixed, we also measure out-of-distribution generalization to other prediction spans at test time (horizontal axes in Figure 2). The test-time prediction spans increase up to the same length as the input sequences, which is 60 for SimP and 24 for RealP. The MSE increases significantly once the testing prediction span exceeds the training span, but still remains low for both cases (note that the vertical axis scale is on the order of 10^{-2}).

In contrast with the motion encoder, the vision encoder is trained to compress and then reconstruct the current video clip rather than predicting observations in the future. Therefore, the out-of-distribution generalization tests above are not applicable here. Instead, there is a single MSE that measures how well the vision encoder reconstructs its input clip. At the end of training, the average squared error per frame (summed over all pixels in the frame) was 5.495, for pixel values in $[0, 1]$, on the held-out test set.

Predicted joint angles are shown in Figure 3. Not all joints are predicted perfectly, but this does not substantially harm the final walk vs. fall prediction accuracy. We hypothesize that even without a perfect motion predictor, the self-supervised training objective results in latent representations with enough information for effective fall prediction.

Layer	Output Channels	Kernel Size	Stride	Padding	Output Padding
`conv1`	64	(2,3,3)	(1,2,2)	(0,1,1)	-
`conv1_branch_1`	256	1	1	0	-
`conv1_branch_2`	128	1	1	0	-
	128	3	1	1	-
	256	1	1	0	-
`conv2`	128	1	1	0	-
	128	3	1	1	-
	256	1	1	0	-
`conv3_branch_1`	512	(3,1,1)	(1,2,2)	0	-
`conv3_branch_2`	128	(3,1,1)	(1,2,2)	0	-
	128	3	1	1	-
	512	1	1	0	-
`conv4_branch_1`	256	(3,1,1)	2	0	-
`conv4_branch_2`	256	(3,1,1)	2	(1,0,0)	-
	256	3	1	1	-
	256	(2,1,1)	1	0	-
`convt3d_1`	512	3	2	1	(0,1,1)
`convt3d_2`	256	3	2	1	1 or (1,0,1)
`convt3d_3`	256	3	2	(0,1,1) or 1	1
`convt3d_4`	64	3	(1,2,2)	1	(0,1,1)
`convt3d_5`	64	(4,3,3) or 3	(1,2,2)	1	(0,1,1)
`convt3d_6`	3	3	2	1	1
`convt3d_7`	3	3	2 or (1,2,2)	1	1 or (0,1,1)

Table 1: Hyper-parameters of our vision model. Different dimensions are organized into a tuple. Otherwise, a single value is used as a shorthand. The extracted features length are 256 for both datasets. The image sizes slightly differ (96×128 for the RealP dataset and 128×128 for the SimP dataset). Corresponding hyper-parameters are listed following a *SimP* or *RealP* format. For example, "(4,3,3) or 3" means the kernel sizes are (4,3,3) for the model trained on the SimP dataset and are all 3 on the RealP dataset.

Appendix D: T-test on Modalities

Our hypothesis was that including all modalities would achieve statistically significantly better performance than any subset of modalities. This was largely supported by SimP, but not RealP, where MTV mean accuracy was not the highest. For SimP, we measured statistical significance with a series of one-sided Welch's t-tests, comparing MVT or MVT* against their respective modality ablations. The p-values of these tests on the SimP dataset are shown in Tables 2 and 3. Most ablations were significant except removing T and in some cases V.

Model	Prediction spans 10	20	30	40	50	60
V	1.4e-13	4.5e-13	4.0e-12	2.3e-13	7.9e-14	5.2e-12
M	0.0005	0.0039	0.0016	0.0006	0.0032	0.0030
TV	1.4e-13	4.5e-13	4.0e-12	2.3e-13	7.9e-14	5.2e-12
MT	1.1e-6	0.0011	4.0e-5	6.8e-5	0.0004	0.0003
MV	<u>0.7879</u>	<u>0.2655</u>	<u>0.2589</u>	<u>0.2552</u>	<u>0.3931</u>	<u>0.5373</u>

Table 2: t-test p-values for MTV compared with each ablation in the same prediction span. Insignificant p-values are <u>underlined</u>.

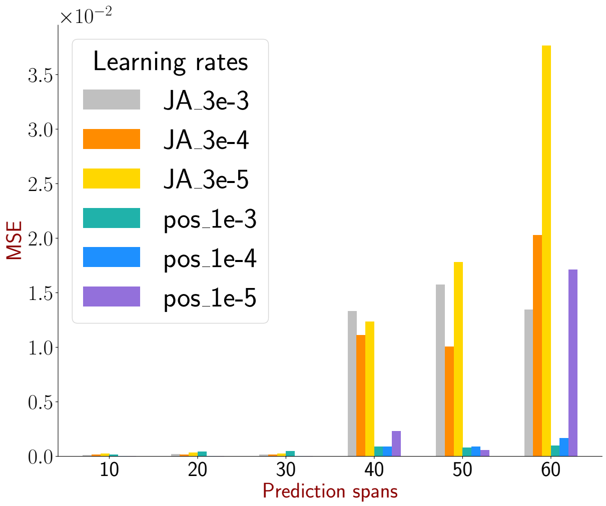
Model	Prediction spans 10	20	30	40	50	60
V*	1.6e-11	3.8e-10	8.3e-10	2.4e-10	1.7e-10	8.6e-10
M*	0.0053	0.0326	0.0308	0.0346	0.0283	0.0211
TV*	1.6e-11	3.8e-10	8.3e-10	2.4e-10	1.7e-10	8.6e-10
MT*	0.0373	<u>0.0681</u>	<u>0.0766</u>	<u>0.0916</u>	<u>0.0988</u>	<u>0.1100</u>
MV*	<u>0.3625</u>	<u>0.2849</u>	<u>0.2404</u>	<u>0.2046</u>	<u>0.1845</u>	<u>0.2160</u>

Table 3: Same as Table 2 but for MTV*.

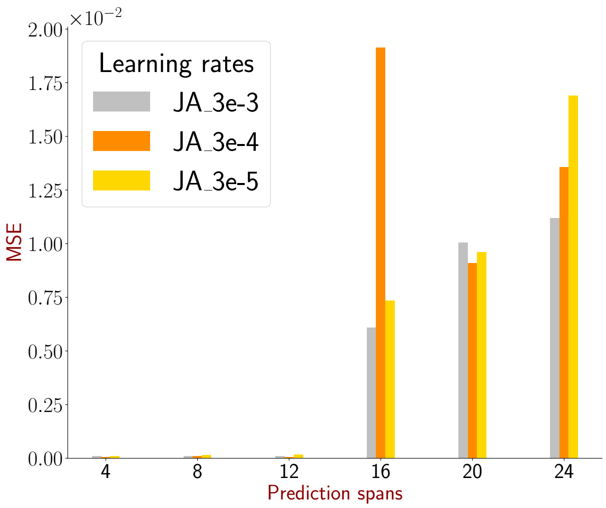
Appendix E: Additional Experiments with EGOFALLS

We conducted additional experimental variations with EGOFALLS to better understand its performance, especially the low accuracy of 60% in recognition mode on the RealP dataset versus more competitive accuracy in prediction mode. In this section, we further explore the reason behind this performance discrepancy, and verify our hypotheses with two sets of experiments.

Our RealP dataset is composed of 20 falling episodes and 10 walking episodes, and each episode contains 300 frames. The original EGOFALLS configuration, which we used for recognition mode in the main paper, samples 10 equally-spaced frames from a complete video for feature extraction,



(a) Testing results on the SimP dataset



(b) Testing results on the RealP dataset

Figure 2: MSE of prediction, on SimP and RealP datasets, for the positions ("pos" denoting local coordinates) and joint angles ("JA").

which constitutes only $1/30^{th}$ of the available frames. Since it is in recognition (and not prediction) mode, it outputs a single label for the entire video. In our main paper results, if a fall happened anywhere during the episode, we labeled it as a "fall" for the purposes of the recognition task.

Our main hypothesis to explain the 60% accuracy is that EGOFALLS fails to capture strong features when its input frames are not heavily sampled from the short interval when a fall actually occurs. Since we sampled 10 equally-spaced frames from the entire episode, few or none of those frames will lie in the short interval when the fall actually occurs. To test this hypothesis, we adopt a shorter time interval for sampling, and still extract 10 frames per video. Specifically, we consider experimental conditions where EGOFALLS takes

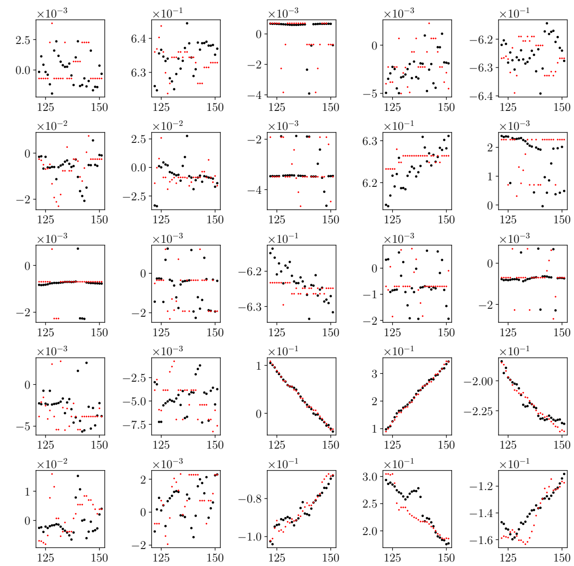


Figure 3: An example of the predicted joint angles for prediction span 12. The horizontal axis refers to time steps and the vertical axis refers to the joint angles in radius. Black dots represent the predictions while red dots represent the ground truth. One panel for each joint.

10 evenly-spaced samples from the first N frames of the episode, where $N \leq 300$ and is fixed at one of the values shown in Table 4. In this case, the video is labeled as "fall" only if a fall occurred during the first N frames. As seen in the table, most falls in the full dataset occur in the last 30 frames, so once $N \leq 270$ the number of examples labeled as falls is reduced. We observe that accuracy increases as N decreases, but this may be due to the greater imbalance in the labels. We did not consider $N \leq 150$ because no falls occur before the 150^{th} frame of any episode.

	Accuracy	# of falling	# of walking
N180	71.94	7	23
N210	76.94	7	23
N240	71.94	7	23
N270	76.94	7	23
N300	60.00	20	10

Table 4: EGOFALLS with shorter time interval. The number behind a leading "N" refers to the maximum length of each video a model can access. For example, N180 means only the first 180 frames are available, N300 refers to the accessibility to the whole video, which is the original configuration.

Having observed that most falls occur near the end of the episodes, we conducted a similar experiment using the **last** L frames, rather than the **first** N frames, where also $L \leq 300$. In this setting, all episodes retain the same labels as the full dataset, because the tail portion of any full episode with a fall will include frames during the fall. The corresponding results in Table 5 show that falling and walking labels match the full RealP dataset. Since RealP uses the real robotic hardware, most falling episodes are terminated soon after a fall is detected to prevent hardware damage. This implies that the time when a fall occurs is close to the end of a video. Accordingly, we observe that the shorter the length L, the higher the accuracy. This supports our hypothesis that heavily sampling frames in the period after a fall can increase the accuracy for EGOFALLS in recognition mode. Since the default configuration samples 10 frames evenly spaced from the full video, and most falls in RealP happen near the end of the full video, the default configuration has lower performance.

	Accuracy	# of falling	# of walking
L300	60.00	20	10
L240	69.17	20	10
L180	74.17	20	10
L120	79.17	20	10
L60	85.00	20	10

Table 5: EGOFALLS with postponed starts. The number behind a leading "L" refers to the length of each video a model can access. For example, L60 means only the last 60 frames are available, L300 refers to the accessibility to the whole video, which is the original configuration.

The foregoing experiments explain low 60% accuracy on the RealP dataset, but the question remains why EGOFALLS in its default configuration performed better on the SimP dataset. Unlike RealP, when generating the simulated dataset, we do not terminate the episode early even when a fall occurs. This means that fall episodes will contain a long tail after the fall occurred. Consequently, when sampling 10 evenly spaced frames from the full video, more of those frames will lie in the portion of the simulation after a fall occurred, and therefore EGOFALLS can capture sufficient features from videos in our SimP dataset, which is unfair to other forecasting models including ours. Accordingly, we did not observe a performance gap between prediction and recognition modes on our SimP dataset.

Overall, equally spaced frames result in a uniform distribution over the whole video that may lead to a performance drop when the fall event time is very **late** in the episode. Our supplementary experiments verify this hypothesis and prove that alternative sampling strategies can more effectively utilize the full episodes and result in higher performance for fall recognition.